one's complement A notation that represents the most negative value by $10\ldots000_{two}$ and the most positive value by $01\ldots11_{two}$, leaving an equal number of negatives and positives but ending up with two zeros, one positive ($00\ldots00_{two}$) and one negative ($11\ldots11_{two}$). The term is also used to mean the inversion of every bit in a pattern: 0 to 1 and 1 to 0.

opcode The field that denotes the operation and format of an instruction.

OpenMP An API for shared memory multiprocessing in C, C++, or Fortran that runs on UNIX and Microsoft platforms. It includes compiler directives, a library, and runtime directives.

OR A logical bit-by-bit operation with two operands that calculates a 1 if there is a 1 in *either* operand.

out-of-order execution A situation in pipelined execution when an instruction blocked from executing does not cause the following instructions to wait.

output device A mechanism that conveys the result of a computation, such as a display, to a user or to another computer.

page fault An event that occurs when an accessed page is not present in main memory.

page table The table containing the virtual to physical address translations in a virtual memory system. The table, which is stored in memory, is typically indexed by the virtual page number; each entry in the table contains the physical page number for that virtual page if the page is currently in memory.

parallel processing program A single program that runs on multiple processors simultaneously.

PC-relative addressing An addressing regime in which the address is the sum of the program counter (PC) and a constant in the instruction.

personal computer (PC) A computer designed for use by an individual, usually incorporating a graphics display, a keyboard, and a mouse.

personal mobile devices (PMDs) Small wireless devices to connect to the Internet; they rely on batteries for power, and software is installed by downloading apps. Conventional examples are smart phones and tablets.

physical address An address in main memory.

physically addressed cache A cache that is addressed by a physical address.

pipeline stall Also called **bubble**. A stall initiated in order to resolve a hazard.

pipelining An implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.

pixel The smallest individual picture element. Screens are composed of hundreds of thousands to millions of pixels, organized in a matrix.

pop Remove element from stack.

precise interrupt Also called **precise exception**. An interrupt or exception that is always associated with the correct instruction in pipelined computers.

prefetching A technique in which data blocks needed in the future are brought into the cache early by the use of special instructions that specify the address of the block.

procedure A stored subroutine that performs a specific task based on the parameters with which it is provided.

procedure call frame A block of memory that is used to hold values passed to a procedure as arguments, to save registers that a procedure may modify but that the procedure's caller does not want changed, and to provide space for variables local to a procedure.

procedure frame Also called **activation record**. The segment of the stack containing a procedure's saved registers and local variables.

process Includes one or more threads, the address space, and the operating system state. Hence, a process switch usually invokes the operating system, but not a thread switch.

program counter (PC) The register containing the address of the instruction in the program being executed.

programmable array logic (PAL) Contains a programmable and-plane followed by a fixed or-plane.

programmable logic array (PLA) A structured-logic element composed of a set of inputs and corresponding input complements and two stages of logic, the first generating product terms of the inputs and input complements, and the second generating sum terms of the product terms. Hence, PLAs implement logic functions as a sum of products.

programmable logic device (PLD) An integrated circuit containing combinational logic whose function is configured by the end user.

programmable ROM (PROM) A form of read-only memory that can be programmed when a designer knows its contents.

propagation time The time required for an input to a flip-flop to propagate to the outputs of the flip-flop.

protection A set of mechanisms for ensuring that multiple processes sharing the processor, memory, or I/O devices cannot interfere, intentionally or unintentionally, with one another by reading or writing each other's data. These mechanisms also isolate the operating system from a user process.

pseudoinstruction A common variation of assembly language instructions often treated as if it were an instruction in its own right.

Pthreads A UNIX API for creating and manipulating threads. It is structured as a library.

push Add element to stack.

quotient The primary result of a division; a number that when multiplied by the divisor and added to the remainder produces the dividend.

read-only memory (ROM) A memory whose contents are designated at creation time, after which the contents can only be read. ROM is used as structured logic to implement a set of logic functions by using the terms in the logic functions as address inputs and the outputs as bits in each word of the memory.

receive message routine A routine used by a processor in machines with private memories to accept a message from another processor.

recursive procedures Procedures that call themselves either directly or indirectly through a chain of calls.

reduction A function that processes a data structure and returns a single value.

reference bit Also called **use bit** or **access bit**. A field that is set whenever a page is accessed and that is used to implement LRU or other replacement schemes.

reg In Verilog, a register.

register file A state element that consists of a set of registers that can be read and written by supplying a register number to be accessed.

register renaming The renaming of registers by the compiler or hardware to remove antidependences.

register use convention Also called **procedure call convention**. A software protocol governing the use of registers by procedures.

relocation information The segment of a UNIX object file that identifies instructions and data words that depend on absolute addresses.

remainder The secondary result of a division; a number that when added to the product of the quotient and the divisor produces the dividend.

reorder buffer The buffer that holds results in a dynamically scheduled processor until it is safe to store the results to memory or a register.

reservation station A buffer within a functional unit that holds the operands and the operation.

response time Also called **execution time**. The total time required for the computer to complete a task, including disk accesses, memory accesses, I/O activities, operating

system overhead, CPU execution time, and so on.

restartable instruction An instruction that can resume execution after an exception is resolved without the exceptions affecting the result of the instruction.

return address A link to the calling site that allows a procedure to return to the proper address; in LEGv8, it is stored in register LR (X30).

rotational latency Also called **rotational delay**. The time required for the desired sector of a disk to rotate under the read/write head; usually assumed to be half the rotation time.

round Method to make the intermediate floating-point result fit the floating-point format; the goal is typically to find the nearest number that can be represented in the format. It is also the name of the second of two extra bits kept on the right during intermediate floating-point calculations, which improves rounding accuracy.

scientific notation A notation that renders numbers with a single digit to the left of the decimal point.

secondary memory Nonvolatile memory used to store programs and data between runs; typically consists of flash memory in PMDs and magnetic disks in servers.

sector One of the segments that make up a track on a magnetic disk; a sector is the smallest amount of information that is read or written on a disk.

seek The process of positioning a read/write head over the proper track on a disk.

segmentation A variable-size address mapping scheme in which an address consists of two parts: a segment number, which is mapped to a physical address, and a segment offset.

selector value Also called **control value**. The control signal that is used to select one of the input values of a multiplexor as the output of the multiplexor.

semiconductor A substance that does not conduct electricity well.

send message routine A routine used by a processor in machines with private memories to pass a message to another processor.

sensitivity list The list of signals that specifies when an always block should be re-evaluated.

separate compilation Splitting a program across many files, each of which can be compiled without knowledge of what is in the other files.

sequential logic A group of logic elements that contain memory and hence whose value depends on the inputs as well as the current contents of the memory.

server A computer used for running larger programs for multiple users, often simultaneously, and typically accessed only via a network.

set-associative cache A cache that has a fixed number of locations (at least two) where each block can be placed.

setup time The minimum time that the input to a memory device must be valid before the clock edge.

shared memory multiprocessor (SMP) A parallel processor with a single physical address space.

sign-extend Increases the size of a data item by replicating the high-order sign bit of the original data item in the high-order bits of the larger, destination data item.

silicon A natural element that is a semiconductor.

silicon crystal ingot A rod composed of a silicon crystal that is between 8 and 12 inches in diameter and about 12 to 24 inches long.

SIMD or Single Instruction stream, Multiple Data streams. The same instruction is applied to many data streams, as in a vector processor.

simple programmable logic device (SPLD) Programmable logic device, usually containing either a single PAL or PLA.

simultaneous multithreading (SMT) A version of multithreading that lowers

the cost of multithreading by utilizing the resources needed for multiple issue, dynamically scheduled microarchitecture.

single precision A floating-point value represented in a 32-bit word.

single-cycle implementation Also called **single clock cycle implementation**. An implementation in which an instruction is executed in one clock cycle. While easy to understand, it is too slow to be practical.

SISD or Single Instruction stream, Single Data stream. A uniprocessor.

Software as a Service (SaaS) delivers software and data as a service over the Internet, usually via a thin program such as a browser that runs on local client devices, instead of binary code that must be installed, and runs wholly on that device. Examples include web search and social networking.

source language The high-level language in which a program is originally written.

spatial locality The locality principle stating that if a data location is referenced, data locations with nearby addresses will tend to be referenced soon.

speculation An approach whereby the compiler or processor guesses the outcome of an instruction to remove it as a dependence in executing other instructions.

split cache A scheme in which a level of the memory hierarchy is composed of two independent caches that operate in parallel with each other, with one handling instructions and one handling data.

SPMD Single Program, Multiple Data streams. The conventional MIMD programming model, where a single program runs across all processors.

stack A data structure for spilling registers organized as a last-in-first-out queue.

stack pointer A value denoting the most recently allocated address in a stack that shows where registers should be spilled or where old register values can be found. In LEGv8, it is register SP.

stack segment The portion of memory used by a program to hold procedure call frames.

state element A memory element, such as a register or a memory.

static data The portion of memory that contains data whose size is known to the compiler and whose lifetime is the program's entire execution.

static multiple issue An approach to implementing a multiple-issue processor where many decisions are made by the compiler before execution.

static random access memory (SRAM) A memory where data are stored statically (as in flip-flops) rather than dynamically (as in DRAM). SRAMs are faster than DRAMs, but less dense and more expensive per bit.

sticky bit A bit used in rounding in addition to guard and round that is set whenever there are nonzero bits to the right of the round bit.

stored-program concept The idea that instructions and data of many types can be stored in memory as numbers and thus be easy to change, leading to the stored program computer.

strong scaling Speed-up achieved on a multiprocessor without increasing the size of the problem.

structural hazard When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute.

structural specification Describes how a digital system is organized in terms of a hierarchical connection of elements.

sum of products A form of logical representation that employs a logical sum (OR) of products (terms joined using the AND operator).

supercomputer A class of computers with the highest performance and cost; they are configured as servers and typically cost tens to hundreds of millions of dollars.

superscalar An advanced pipelining technique that enables the processor to execute more than one instruction per clock cycle by selecting them during execution.

supervisor mode Also called **kernel mode**. A mode indicating that a running process is an operating system process.

swap space The space on the disk reserved for the full virtual memory space of a process.

symbol table A table that matches names of labels to the addresses of the memory words that instructions occupy.

synchronization The process of coordinating the behavior of two or more processes, which may be running on different processors.

synchronizer failure A situation in which a flip-flop enters a metastable state and where some logic blocks reading the output of the flip-flop see a 0 while others see a 1.

synchronous system A memory system that employs clocks and where data signals are read only when the clock indicates that the signal values are stable.

system call A special instruction that transfers control from user mode to a dedicated location in supervisor code space, invoking the exception mechanism in the process.

system CPU time The CPU time spent in the operating system performing tasks on behalf of the program.

systems software Software that provides services that are commonly useful, including operating systems, compilers, loaders, and assemblers.

tag A field in a table used for a memory hierarchy that contains the address information required to identify whether the associated block in the hierarchy corresponds to a requested word.

task-level parallelism or process-level parallelism Utilizing multiple processors by running independent programs simultaneously.

temporal locality The principle stating that if a data location is referenced, then it will tend to be referenced again soon.

terabyte (TB) Originally 1,099,511,627,776 (240) bytes, although communications and secondary storage systems developers started using the term to mean 1,000,000,000,000 (10^{12}) bytes. To reduce confusion, we now use the term **tebibyte** (TiB) for 240 bytes, defining terabyte (TB) to mean 10^{12} bytes. (Figure 1.1 shows the full range of decimal and binary values and names.)

text segment The segment of a UNIX object file that contains the machine language code for routines in the source file.

thread A thread includes the program counter, the register state, and the stack. It is a lightweight process; whereas threads commonly share a single address space, processes don't.

three Cs model A cache model in which all cache misses are classified into one of three categories: compulsory misses, capacity misses, and conflict misses.

throughput Also called **bandwidth**. Another measure of performance, it is the number of tasks completed per unit time.

tournament branch predictor A branch predictor with multiple predictions for each branch and a selection mechanism that chooses which predictor to enable for a given branch.

track One of thousands of concentric circles that makes up the surface of a magnetic disk.

transistor An on/off switch controlled by an electric signal.

translation-lookaside buffer (TLB) A cache that keeps track of recently used address mappings to try to avoid an access to the page table.

truth table From logic, a representation of a logical operation by listing all the values of the inputs and then in each case showing what the resulting outputs should be.

underflow (floating-point) A situation in which a negative exponent becomes too large to fit in the exponent field.

uniform memory access (UMA) A multiprocessor in which latency to any word in main memory is about the same no matter which processor requests the access.

units in the last place (ulp) The number of bits in error in the least significant bits of the significand between the actual number and the number that can be represented.

unmapped A portion of the address space that cannot have page faults.

unresolved reference A reference that requires more information from an outside source to be complete.

use bit Also called **reference bit** or **access bit**. A field that is set whenever a page is accessed and that is used to implement LRU or other replacement schemes.

use latency Number of clock cycles between a load instruction and an instruction that can use the result of the load without stalling the pipeline.

user CPU time The CPU time spent in a program itself.

valid bit A field in the tables of a memory hierarchy that indicates that the associated block in the hierarchy contains valid data.

vector lane One or more vector functional units and a portion of the vector register file. Inspired by lanes on highways that increase traffic speed, multiple lanes execute vector operations simultaneously.

vectored interrupt An interrupt for which the address to which control is transferred is determined by the cause of the exception.

Verilog One of the two most common hardware description languages.

very-large-scale integrated (VLSI) circuit A device containing hundreds of thousands to millions of transistors.

very long instruction word (VLIW) A style of instruction set architecture that launches many operations that are defined to be independent in a single wide instruction, typically with many separate opcode fields.

VHDL One of the two most common hardware description languages.

virtual address An address that corresponds to a location in virtual space and is translated by address mapping to a physical address when memory is accessed.

virtual machine A virtual computer that appears to have nondelayed branches and loads and a richer instruction set than the actual hardware.

virtual memory A technique that uses main memory as a "cache" for secondary storage.

virtually addressed cache A cache that is accessed with a virtual address rather than a physical address.

volatile memory Storage, such as DRAM, that retains data only if it is receiving power.

wafer A slice from a silicon ingot no more than 0.1 inches thick, used to create chips.

weak scaling Speed-up achieved on a multiprocessor while expanding the size of the problem proportionally to the increase in the number of processors.

wide area network (WAN) A network extended over hundreds of kilometers that can span a continent.

wire In Verilog, specifies a combinational signal.

word The natural unit of access in a computer, usually a group of 32 bits.

workload A set of programs run on a computer that is either the actual collection of applications run by a user or constructed from real programs to approximate such a mix. A typical workload specifies both the programs and the relative frequencies.

write buffer A queue that holds data while the data are waiting to be written to memory.

write-back A scheme that handles writes by updating values only to the block in the cache, then writing the modified block to the lower level of the hierarchy when the block is replaced.

write-through A scheme in which writes always update both the cache and the next lower level of the memory hierarchy, ensuring that data are always consistent between the two.

yield The percentage of good dies from the total number of dies on the wafer.

Further Reading

Chapter 1

Barroso, L. and U. Hölzle [2007]. "The case for energy-proportional computing", *IEEE Computer*, December.

A plea to change the nature of computer components so that they use much less power when lightly utilized.

Bell, C. G. [1996]. *Computer Pioneers and Pioneer Computers*, ACM and the Computer Museum, videotapes.

Two videotapes on the history of computing, produced by Gordon and Gwen Bell, including the following machines and their inventors: Harvard Mark-I, ENIAC, EDSAC, IAS machine, and many others.

Burks, A.W., H.H. Goldstine, and J. von Neumann [1946]. "Preliminary discussion of the logical design of an electronic computing instrument," *Report to the U.S. Army Ordnance Department*, p. 1; also appears in *Papers of John von Neumann*, W. Aspray and A. Burks (Eds.), MIT Press, Cambridge, MA, and Tomash Publishers, Los Angeles, 1987, 97–146.

A classic paper explaining computer hardware and software before the first stored-program computer was built. We quote extensively from it in Chapter 3. It simultaneously explained computers to the world and was a source of controversy because the first draft did not give credit to Eckert and Mauchly.

Campbell-Kelly, M. and W. Aspray [1996]. *Computer: A History of the Information Machine*, Basic Books, New York.

Two historians chronicle the dramatic story. The New York Times *calls it well written and authoritative.*

Ceruzzi, P. F. [1998]. *A History of Modern Computing*, MIT Press, Cambridge, MA.

Contains a good description of the later history of computing: the integrated circuit and its impact, personal computers, UNIX, and the Internet.

Curnow, H. J. and B. A. Wichmann [1976]. "A synthetic benchmark", *The Computer J.* 19(1):80.

Describes the first major synthetic benchmark, Whetstone, and how it was created.

Flemming, P. J. and J. J. Wallace [1986]. "How not to lie with statistics: The correct way to summarize benchmark results", *Commun. ACM* 29(3 (March)), 218–221.

Describes some of the underlying principles in using different means to summarize performance results.

Goldstine, H. H. [1972]. *The Computer: From Pascal to von Neumann*, Princeton University Press, Princeton, NJ.

A personal view of computing by one of the pioneers who worked with von Neumann.

Hayes, B. [2007]. "Computing in a parallel universe", *American Scientist*, Vol. 95(November–December), 476–480.

An overview of the parallel computing challenge written for the layman.

Hennessy, J. L. and D. A. Patterson [2012]. *Chapter 1 of Computer Architecture: A Quantitative Approach*, fifth edition, Morgan Kaufmann Publishers, Waltham, MA.

Section 1.5 goes into more detail on power, Section 1.6 contains much more detail on the cost of integrated circuits and explains the reasons for the difference between price and cost, and Section 1.8 gives more details on evaluating performance.

Lampson, B.W. [1986]. "Personal distributed computing; The Alto and Ethernet software." In ACM Conference on the History of Personal Workstations (January).

Thacker, C.R. [1986]. "Personal distributed computing: The Alto and Ethernet hardware," In ACM Conference on the History of Personal Workstations (January).

These two papers describe the software and hardware of the landmark Alto.

Metropolis, N., J. Howlett, and G. -C. Rota (Eds.) [1980]. *A History of Computing in the Twentieth Century*, Academic Press, New York.

A collection of essays that describe the people, software, computers, and laboratories involved in the first experimental and commercial computers. Most of the authors were personally involved in the projects. An excellent bibliography of early reports concludes this interesting book.

Public Broadcasting System [1992]. *The Machine That Changed the World*, videotapes.

These five 1-hour programs include rare footage and interviews with pioneers of the computer industry.

Slater, R. [1987]. *Portraits in Silicon*, MIT Press, Cambridge, MA.

Short biographies of 31 computer pioneers.

Stern, N. [1980]. "Who invented the first electronic digital computer?" *Annals of the History of Computing* 2:4 (October), 375–376.

A historian s perspective on Atanasoff versus Eckert and Mauchly.

Wilkes, M. V. [1985]. *Memoirs of a Computer Pioneer*, MIT Press, Cambridge, MA.

A personal view of computing by one of the pioneers.

Chapter 2

Bayko, J. [1996]. "Great microprocessors of the past and present," search for it on the *http://www.cpushack.com/CPU/cpu.html*.

A personal view of the history of both representative and unusual microprocessors, from the Intel 4004 to the Patriot Scientific ShBoom!

Kane, G. and J. Heinrich [1992]. *MIPS RISC Architecture*, Prentice Hall, Englewood Cliffs, NJ.

This book describes the MIPS architecture in greater detail than Appendix A.

Levy, H. and R. Eckhouse [1989]. *Computer Programming and Architecture: The VAX*, Digital Press, Boston.

This book concentrates on the VAX, but also includes descriptions of the Intel 8086, IBM 360, and CDC 6600.

Morse, S., B. Ravenal, S. Mazor, and W. Pohlman [1980]. "Intel microprocessors—8080 to 8086", *Computer* 13:10 (October).

The architecture history of the Intel from the 4004 to the 8086, according to the people who participated in the designs.

Wakerly, J. [1989]. *Microcomputer Architecture and Programming*, Wiley, New York.

The Motorola 6800 is the main focus of the book, but it covers the Intel 8086, Motorola 6809, TI 9900, and Zilog Z8000.

Chapter 3

If you are interested in learning more about floating point, two publications by David Goldberg [1991, 2002] are good starting points; they abound with pointers to further reading. Several of the stories told in this section come from Kahan [1972, 1983]. The latest word on the state of the art in computer arithmetic is often found in the *Proceedings* of the latest IEEE-sponsored Symposium on Computer Arithmetic, held every 2 years; the 16th was held in 2003.

Burks, A.W., H.H. Goldstine, and J. von Neumann [1946]. "Preliminary discussion of the logical design of an electronic computing instrument," *Report to the U.S. Army Ordnance Dept.*, p. 1; also in *Papers of John von Neumann*, W. Aspray and A. Burks (Eds.), MIT Press, Cambridge, MA, and Tomash Publishers, Los Angeles, 1987, 97–146.

This classic paper includes arguments against floating-point hardware.

Goldberg, D. [2002]. "Computer arithmetic". Appendix J of *Computer Architecture: A Quantitative Approach*, fifth edition, J. L. Hennessy and D. A. Patterson, Morgan Kaufmann Publishers, Waltham, MA.

A more advanced introduction to integer and floating-point arithmetic, with emphasis on hardware. It covers Sections 3.4–3.6 of this book in just 10 pages, leaving another 45 pages for advanced topics.

Goldberg, D. [1991]. "What every computer scientist should know about floating-point arithmetic", *ACM Computing Surveys* 23(1) 5–48.

Another good introduction to floating-point arithmetic by the same author, this time with emphasis on software.

Kahan, W. [1972]. "A survey of error-analysis". *Info. Processing 71 (Proc. IFIP Congress 71 in Ljubljana)*, Vol. 2, North-Holland Publishing, Amsterdam, 1214–1239

This survey is a source of stories on the importance of accurate arithmetic.

Kahan, W. [1983]. "Mathematics written in sand", *Proc. Amer. Stat. Assoc. Joint Summer Meetings of 1983, Statistical Computing Section*, 12–26.

The title refers to silicon and is another source of stories illustrating the importance of accurate arithmetic.

Kahan, W. [1990]. *"On the advantage of the 8087's stack," unpublished course notes, Computer Science Division*, University of California, Berkeley.

What the 8087 floating-point architecture could have been.

Kahan, W. [1997]. Available at http://www.cims.nyu.edu/~dbindel/class/cs279/87stack.pdf.

A collection of memos related to floating point, including "Beastly numbers" (another less famous Pentium bug), "Notes on the IEEE floating point arithmetic" (including comments on how some features are atrophying), and "The baleful effects of computing benchmarks" (on the unhealthy preoccupation on speed versus correctness, accuracy, ease of use, flexibility, …).

Koren, I. [2002]. *Computer Arithmetic Algorithms*, second edition, A. K. Peters, Natick, MA.

A textbook aimed at seniors and first-year graduate students that explains fundamental principles of basic arithmetic, as well as complex operations such as logarithmic and trigonometric functions.

Wilkes, M. V. [1985]. *Memoirs of a Computer Pioneer*, MIT Press, Cambridge, MA.

This computer pioneer's recollections include the derivation of the standard hardware for multiply and divide developed by von Neumann.

Chapter 4

Bhandarkar, D. and D. W. Clark [1991]. "Performance from architecture: Comparing a RISC and a CISC with similar hardware organizations," *Proc. Fourth Conf. on Architectural Support for Programming Languages and Operating Systems*, IEEE/ACM (April), Palo Alto, CA, 310–319.

A quantitative comparison of RISC and CISC written by scholars who argued for CISCs as well as built them; they conclude that MIPS is between 2 and 4 times faster than a VAX built with similar technology, with a mean of 2.7.

Fisher, J. A. and B. R. Rau [1993]. *Journal of Supercomputing (January)*, Kluwer.

This entire issue is devoted to the topic of exploiting ILP. It contains papers on both the architecture and software and is a wonderful source for further references.

Hennessy, J. L. and D. A. Patterson [2012]. *Computer Architecture: A Quantitative Approach*, fifth edition, Morgan Kaufmann, Waltham, MA.

Chapter 3 and Appendix C go into considerably more detail about pipelined processors (almost 200 pages), including superscalar processors and VLIW processors. Appendix G describes Itanium.

Jouppi, N. P. and D. W. Wall [1989]. "Available instruction-level parallelism for superscalar and superpipelined processors," *Proc. Third Conf. on Architectural Support for Programming Languages and Operating Systems*, IEEE/ACM (April), Boston, 272–82.

A comparison of deeply pipelined (also called superpipelined) and superscalar systems.

Kogge, P. M. [1981]. *The Architecture of Pipelined Computers*, McGraw-Hill, New York.

A formal text on pipelined control, with emphasis on underlying principles.

Russell, R. M. [1978]. "The CRAY-1 computer system", *Commun. ACM* 21:1 (January), 63–72.

A short summary of a classic computer that uses vectors of operations to remove pipeline stalls.

Smith, A. and J. Lee [1984]. "Branch prediction strategies and branch target buffer design", *Computer* 17:1 (January), 6–22.

An early survey on branch prediction.

Smith, J. E. and A. R. Plezkun [1988]. "Implementing precise interrupts in pipelined processors", *IEEE Trans. on Computers* 37:5 (May), 562–573.

Covers the difficulties in interrupting pipelined computers.

Thornton, J. E. [1970]. *Design of a Computer: The Control Data 6600*, Scott, Foresman, Glenview, IL.

A classic book describing a classic computer, considered the first supercomputer.

Chapter 5

Cantin, J. F. and M. D. Hill [2001]. "Cache performance for selected SPEC CPU2000 benchmarks", *SIGARCH Computer Architecture News* 29:4 (September), 13–18.

A reference paper of cache miss rates for many cache sizes for the SPEC2000 benchmarks.

Conti, C, D. H. Gibson, and S. H. Pitowsky [1968]. "Structural aspects of the System/360 Model 85, part I: General organization", *IBM Systems J.* 7:1, 2–14.

A classic paper that describes the first commercial computer to use a cache and its resulting performance.

Hennessy, J. and D. Patterson [2012]. *Chapter 2 and Appendix B in Computer Architecture: A Quantitative Approach*, fifth edition, Morgan Kaufmann Publishers, Waltham, MA.

For more in-depth coverage of a variety of topics including protection, cache performance of out-of-order processors, virtually addressed caches, multilevel caches, compiler optimizations, additional latency tolerance mechanisms, and cache coherency.

Kilburn, T., D. B. G Edwards, M. J. Lanigan, and F. H. Sumner [1962]. "One-level storage system," *IRE Transactions on Electronic Computers* EC-11 (April), 223–35. Also appears in D. P. Siewiorek, C G Bell, and A. Newell [1982], *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 135–48.

This classic paper is the first proposal for virtual memory.

LaMarca, A. and R. E. Ladner [1996]. "The influence of caches on the performance of heaps," *ACM J. of Experimental Algorithmics* Vol. 1.

This paper shows the difference between complexity analysis of an algorithm, instruction count performance, and memory hierarchy for four sorting algorithms.

McCalpin, J.D. [1995]. "STREAM: Sustainable Memory Bandwidth in High Performance Computers," https://www.cs.virginia.edu/stream/.

A widely used microbenchmark that measures the performance of the memory system behind the caches.

Przybylski, S. A. [1990]. *Cache and Memory Hierarchy Design: A Performance-Directed Approach*, Morgan Kaufmann Publishers, San Francisco.

A thorough exploration of multilevel memory hierarchies and their performance.

Ritchie, D. [1984]. "The evolution of the UNIX time-sharing system", *AT& T Bell Laboratories Technical Journal* 1984:1577–1593.

The history of UNIX from one of its inventors.

Ritchie, D. M. and K. Thompson [1978]. "The UNIX time-sharing system", *Bell System Technical Journal* (August), 1991–2019.

A paper describing the most elegant operating system ever invented.

Silberschatz, A., P. Galvin, and G. Grange [2003]. *Operating System Concepts*, sixth edition, Addison-Wesley, Reading, MA.

An operating systems textbook with a thorough discussion of virtual memory processes and process management, and protection issues.

Smith, A. J. [1982]. "Cache memories", *Computing Surveys* 14:3 (September), 473–530.

The classic survey paper on caches. This paper defined the terminology for the field and has served as a reference for many computer designers.

Smith, D. K. and R. C. Alexander [1988]. *Fumbling the Future: How Xerox Invented, Then Ignored, the First Personal Computer*, Morrow, New York.

A popular book that explains the role of Xerox PARC in laying the foundation for today's computing, but which Xerox did not substantially benefit from.

Tanenbaum, A. [2001]. *Modern Operating Systems*, second edition, Upper Saddle River: Prentice Hall, NJ.

An operating system textbook with a good discussion of virtual memory.

Wilkes, M. [1965]. "Slave memories and dynamic storage allocation", *IEEE Trans. Electronic Computers EC* 14(2 (April)), 270–271.

The first classic paper on caches.

Chapter 6

Almasi, G. S. and A. Gottlieb [1989]. *Highly Parallel Computing*, Benjamin/Cummings, Redwood City, CA.

A textbook covering parallel computers.

Amdahl, G.M. [1967]. "Validity of the single processor approach to achieving large scale computing capabilities," *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ (April), 483–85.

Written in response to the claims of the Illiac IV, this three-page article describes Amdahl's law and gives the classic reply to arguments for abandoning the current form of computing.

Andrews, G. R. [1991]. Concurrent Programming: Principles and Practice, Benjamin/Cummings, Redwood City, CA.

A text that gives the principles of parallel programming.

Archibald, J. and J.-L. Baer [1986]. "Cache coherence protocols: Evaluation using a multiprocessor simulation model", *ACM Trans. on Computer Systems* 4:4 (November), 273–98.

Classic survey paper of shared-bus cache coherence protocols.

Arpaci-Dusseau, A., R. Arpaci-Dusseau, D. Culler, J. Hellerstein, and D. Patterson [1997]. "High-performance sorting on networks of workstations," *Proc. ACM SIG MOD/PODS Conference on Management of Data*, Tucson, AZ (May), 12–15.

How a world record sort was performed on a cluster, including architecture critique of the workstation and network interface. By April 1, 1997, they pushed the record to 8.6 GB in 1 minute and 2.2 seconds to sort 100 MB.

Asanovic, K., R. Bodik, B. C. Catanzaro, J. J. Gebis, P. Husbands, K. Keutzer, D. A. Patterson, W. L. Plishker, J. Shalf, S. W. Williams, and K. A. Yelick [2006]. "The landscape of parallel computing research: A view from Berkeley". *Tech. Rep. UCB/EECS-2006-183*, EECS Department, University of California, Berkeley. (December 18).

Nicknamed the "Berkeley View," this report lays out the landscape of the multicore challenge.

Bailey, D.H., E. Barszcz, J.T. Barton, D.S. Browning, R.L. Carter, L. Dagum, R.A. Fatoohi, P.O. Frederickson, T.A. Lasinski, R.S. Schreiber, H.D. Simon, V. Venkatakrishnan, and S.K. Weeratunga. [1991]. "The NAS parallel benchmarks—summary and preliminary results," *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing* (August).

Describes the NAS parallel benchmarks.

Bell, C. G. [1985]. "Multis: A new class of multiprocessor computers", *Science* 228(April 26), 462–467.

Distinguishes shared address and nonshared address multiprocessors based on microprocessors.

Bienia, C., S. Kumar, J.P. Singh, and K. Li [2008]. "The PARSEC benchmark suite: characterization and architectural implications," Princeton University Technical Report TR-81 1-008 (January).

Describes the PARSEC parallel benchmarks. Also see http://parsec.cs.princeton.edu/.

Cooper, B.F., A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears [2010]. Benchmarking cloud serving systems with YCSB, In: *Proceedings of the 1st ACM Symposium on Cloud Computing*, Indianapolis, Indiana, USA. http://dx.doi.org/10.1145/1807128.1807152 (June).

Presents the "Yahoo! Cloud Serving Benchmark" (YCSB) framework, with the goal of facilitating performance comparisons of the new generation of cloud data serving systems.

Culler, D. E., J. P. Singh, and A. Gupta [1998]. *Parallel Computer Architecture*, Morgan Kaufmann, San Francisco.

A textbook on parallel computers.

Dongarra, J. J., J. R. Bunch, G. B. Moler, and G. W. Stewart [1979]. *LINPACK Users' Guide*, Society for Industrial Mathematics.

The original document describing Linpack, which became a widely used parallel benchmark.

Falk, H. [1976]. "Reaching for the gigaflop", *IEEE Spectrum* 13:10 (October), 65–70.

Chronicles the sad story of the Illiac IV: four times the cost and less than one-tenth the performance of original goals.

Flynn, M. J. [1966]. "Very high-speed computing systems", *Proc. IEEE* 54:12 (December), 1901–09.

Classic article showing SISD/SIMD/MISD/MIMD classifications.

Hennessy, J. and D. Patterson [2012]. "Chapters 5 and Appendices F and I". In *Computer Architecture: A Quantitative Approach*, fifth edition, Morgan Kaufmann Publishers, Waltham, MA.

A more in-depth coverage of a variety of multiprocessor and cluster topics, including programs and measurements.

Henning, J. L. [2007]. "SPEC CPU suite growth: an historical perspective", *Computer Architecture News* Vol. 35, no. 1 (March).

Gives the history of SPEC, including the use of SPECrate to measure performance on independent jobs, which is being used as a parallel benchmark.

Hord, R. M. [1982]. *The Illiac-IV, the First Supercomputer*, Computer Science Press, Rockville, MD.

A historical accounting of the Illiac IV project.

Hwang, K. [1993]. *Advanced Computer Architecture with Parallel Programming*, McGraw-Hill, New York.

Another textbook covering parallel computers.

Kozyrakis, C. and D. Patterson [2003]. "Scalable vector processors for embedded systems", *IEEE Micro* 23:6 (November–December), 36–45.

Examination of a vector architecture for the MIPS instruction set in media and signal processing.

Laprie, J.-C. [1985]. "Dependable computing and fault tolerance: Concepts and terminology," *15th Annual Int'l Symposium on Fault-Tolerant Computing FTCS 15*, Digest of Papers, Ann Arbor, MI (June 19–21) 2–11.

The paper that introduced standard definitions of dependability, reliability, and availability.

Menabrea, L. F. [1842]. "Sketch of the analytical engine invented by Charles Babbage", *Bibliothèque Universelle de Genève* (October).

Certainly the earliest reference on multiprocessors, this mathematician made this comment while translating papers on Babbage's mechanical computer.

Patterson, D., G. Gibson, and R. Katz [1988]. "A case for redundant arrays of inexpensive disks (RAID)," *SIGMOD Conference*, 109–16.

Pfister, G. F. [1998]. *In Search of Clusters: The Coming Battle in Lowly Parallel Computing*, second edition, Prentice Hall, Upper Saddle River, NJ.

An entertaining book that advocates clusters and is critical of NUMA multiprocessors.

Regnier, G., S. Makineni, R. Illikkal, R. Iyer, D. Minturn, R. Huggahalli, D. Newell, L. Cline, and A. Foong [2004]. TCP onloading for data center servers. IEEE Computer, 37(11):48–58.

Describes the work of researchers at Intel Labs, who have experimented with alternative solutions that improve the server's ability to process TCP/IP packets efficiently and at very high rates.

Seitz, C. [1985]. "The Cosmic Cube", *Comm. ACM* 28:1 (January), 22–31.

A tutorial article on a parallel processor connected via a hypertree. The Cosmic Cube is the ancestor of the Intel supercomputers.

Slotnick, D. L. [1982]. "The conception and development of parallel processors—a personal memoir", *Annals of the History of Computing* 4:1 (January), 20–30.

Recollections of the beginnings of parallel processing by the architect of the Illiac IV.

Williams, S., A. Waterman, and D. Patterson [2009]. "Roofline: An insightful visual performance model for multicore architectures", *Communications of the ACM*, 52:4 (April), 65–76.

Williams, S., J. Carter, L. Oliker, J. Shalf, and K. Yelick [2008]. "Lattice Boltzmann simulation optimization on leading multicore platforms," *International Parallel & Distributed Processing Symposium (IPDPS)*.

Paper containing the results of the four multicores for LBMHD.

Williams, S., L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel [2007]. "Optimization of sparse matrix-vector multiplication on emerging multicore platforms", *Supercomputing (SC)*

Paper containing the results of the four multicores for SPmV.

Williams, S. [2008]. *Autotuning Performance of Multicore Computers*, Ph.D. Dissertation, U.C. Berkeley.

Dissertation containing the roofline model.

Woo, S.C., M. Ohara, E. Torrie, J.P. Singh, and A. Gupta. "The SPLASH-2 programs: characterization and methodological considerations," *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA '95)*, May, 24–36.

Paper describing the second version of the Stanford parallel benchmarks.

Appendix A

There are a number of good texts on logic design. Here are some you might like to look into.

Ashenden, P. [2007]. *Digital Design: An Embedded Systems Approach Using VHDL/Verilog*, Waltham, MA: Morgan Kaufmann.

Ciletti, M. D. [2002]. *Advanced Digital Design with the Verilog HDL*, Englewood Cliff s, NJ: Prentice Hall.

A thorough book on logic design using Verilog.

Harris, D. and S. Harris [2012]. *Digital Design and Computer Architecture*, Waltham, MA: Morgan Kaufmann.

A unique and modern approach to digital design using VHDL and SystemVerilog.

Katz, R. H. [2004]. *Modern Logic Design*, 2nd ed., Reading, MA: Addison-Wesley.

A general text on logic design.

Wakerly, J. F. [2000]. *Digital Design: Principles and Practices*, 3rd ed., Englewood Cliff s, NJ: Prentice Hall.

A general text on logic design.

Appendix B

Akeley, K. and T. Jermoluk [1988]. "High-Performance Polygon Rendering," *Proc. SIGGRAPH 1988* (August), 239–46.

Akeley, K. [1993]. "RealityEngine Graphics." *Proc. SIGGRAPH 1993* (August), 109–16.

Blelloch, G. B. [1990]. "Prefix Sums and Their Applications". In John H. Reif (Ed.), *Synthesis of Parallel Algorithms*, Morgan Kaufmann Publishers, San Francisco.

Blythe, D. [2006]. "The Direct3D 10 System", *ACM Trans. Graphics* Vol. 25 no. 3, (July), 724–734.

Buck, I., T. Foley, D. Horn, J. Sugerman, K. Fatahlian, M. Houston, and P. Hanrahan [2004]. "Brook for GPUs: Stream Computing on Graphics Hardware." *Proc. SIGGRAPH 2004*, 777–86, August. http://doi.acm.org/10.1145/1186562.1015800.

Elder, G. [2002] "Radeon 9700." Eurographics/SIGGRAPH Workshop on Graphics Hardware, Hot3D Session, www.graphicshardware.org/previous/www_2002/presentations/Hot3D-RADEON9700.ppt.

Fernando, R. and M. J. Kilgard [2003]. *The Cg Tutorial: The Definitive Guide to Programmable Real-Time Graphics*, Addison-Wesley, Reading, MA.

Fernando, R. (Ed.), [2004]. *GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*, Addison-Wesley, Reading, MA. https://developer.nvidia.com/gpugems/GPUGems/gpugems_pref01.html.

Foley, J., A. van Dam, S. Feiner, and J. Hughes [1995]. *Computer Graphics: Principles and Practice, second edition in C*, Addison- Wesley, Reading, MA.

Hillis, W. D. and G. L. Steele [1986]. "Data parallel algorithms", *Commun. ACM* 29:12 (Dec.), 1170–83. http://doi.acm.org/10.1145/7902.7903.

IEEE 754R Working Group [2006]. *DRAFT Standard for Floating-Point Arithmetic P754*. www.validlab.com/754R/drafts/archive/2006-10-04.pdf.

Industrial Light and Magic [2003]. *OpenEXR*, www.openexr.com.

Intel Corporation [2007]. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. November. Order Number: 248966-016. Also: http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf.

Kessenich, J. [2006]. *The OpenGL Shading Language, Language Version 1.20, Sept. 2006*. www.opengl.org/documentation/specs/.

Kirk, D. and D. Voorhies [1990]. "The Rendering Architecture of the DN10000VS." *Proc. SIGGRAPH* 1990 (August), 299–307.

Lindholm E., M.J. Kilgard, and H. Moreton [2001]. "A User-Programmable Vertex Engine." *Proc. SIGGRAPH* 2001 (August), 149–58.

Lindholm, E., J. Nickolls, S. Oberman, and J. Montrym [2008]. "NVIDIA Tesla: A Unified Graphics and Computing Architecture", *IEEE Micro* Vol. 28, no. 2 (March–April), 39–55.

Microsoft Corporation. *Microsoft DirectX Specification*, https://msdn.microsoft.com/en-us/library/windows/apps/hh452744.aspx.

Microsoft Corporation. [2003]. *Microsoft DirectX 9 Programmable Graphics Pipeline*, Microsoft Press, Redmond, WA.

Montrym, J., D. Baum, D. Dignam, and C. Migdal [1997]. "InfiniteReality: A Real-Time Graphics System." *Proc. SIGGRAPH 1997* (August), 293–301.

Montrym, J. and H. Moreton [2005]. "The GeForce 6800", *IEEE Micro* Vol. 25, no. 2 (March–April), 41–51.

Moore, G. E. [1965]. "Cramming more components onto integrated circuits", *Electronics*, Vol. 38, no. 8 (April 19).

Nguyen, H. ed. [2008]. *GPU Gems 3*, Addison-Wesley, Reading, MA.

Nickolls, J., I. Buck, M. Garland, and K. Skadron [2008]. "Scalable Parallel Programming with CUDA", *ACM Queue*, Vol. 6, no. 2 (March–April), 40–53.

NVIDIA [2007]. CUDA Zone. http://www.nvidia.com/object/cuda_home_new.html.

NVIDIA [2007]. *CUDA Programming Guide 1.1*. https://developer.nvidia.com/nvidia-gpu-programming-guide.

NVIDIA [2007]. *PTX: Parallel Thread Execution ISA version 1.1*. www.nvidia.com/object/io_1195170102263.html.

Nyland, L., M. Harris, and J. Prins [2007]. "Fast N-Body Simulation with CUDA". In *GPU Gems 3*, H. Nguyen (Ed.), Addison-Wesley, Reading, MA.

Oberman, S.F. and M.Y. Siu [2005]. "A High-Performance Area-Efficient Multifunction Interpolator," *Proc. Seventeenth IEEE Symp. Computer Arithmetic*, 272–79.

Pharr, M. (Ed.), [2005]. *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*, Addison-Wesley, Reading, MA.

Satish, N., M. Harris, and M. Garland [2008]. "Designing Efficient Sorting Algorithms for Manycore GPUs," NVIDIA Technical Report NVR-2008-001.

Segal, M. and K. Akeley [2006]. *The OpenGL Graphics System: A Specification, Version 2.1, Dec. 1, 2006*. www.opengl.org/documentation/specs/.

Sengupta, S., M. Harris, Y. Zhang, and J.D. Owens [2007]. "Scan Primitives for GPU Computing." In *Proc. of Graphics Hardware 2007* (August), 97–106.

Volkov, V. and J. Demmel [2008]. "LU, QR and Cholesky Factorizations using Vector Capabilities of GPUs," Technical Report No. UCB/EECS-2008-49, 1–11. http://www.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-49.pdf.

Williams, S., L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel [2007]. "Optimization of sparse matrix-vector multiplication on emerging multicore platforms," In *Proc. Supercomputing 2007*, November.

Appendix D

Bhandarkar, D. P. [1995]. *Alpha Architecture and Implementations*, Newton, MA: Digital Press.

Darcy, J.D., and D. Gay [1996]. "FLECKmarks: Measuring floating point performance using a compliant arithmetic benchmark," CS 252 class project, U.C Berkeley (see *www.sonic.net/~jddarcy/Research/fleckmrk.pdf*)).

Digital Semiconductor. [1996]. *Alpha Architecture Handbook, Version 3*, Digital Press, Maynard, MA. Order number EC-QD2KB-TE (October).

Furber, S. B. [1996]. *ARM System Architecture*, Addison-Wesley, Harlow, England. (See *http://www.pearsonhighered.com/pearsonhigheredus/educator/product/products_detail.page?isbn=9780201675191&forced_logout=forced_logged_out#sthash.QX4WfErc*).

Hewlett-Packard [1994]. *PA-RISC 2.0 Architecture Reference Manual*, 3rd ed.

Hitachi [1997]. *SuperH RISC Engine SH7700 Series Programming Manual*. (See *http://am.renesas.com/products/mpumcu/superh/sh7700/Documentation.jsp*).

IBM. [1994]. *The PowerPC Architecture*, San Francisco: Morgan Kaufmann.

Kane, G. [1996]. *PA-RISC 2.0 Architecture*, Upper Saddle River, NJ: Prentice Hall PTR.

Kane, G. and J. Heinrich [1992]. *MIPS RISC Architecture*, Englewood Cliffs, NJ: Prentice Hall.

Kissell, K.D. [1997]. *MIPS16: High-Density for the Embedded Market*.

Magenheimer, D. J., L. Peters, K. W. Pettis, and D. Zuras [1988]. "Integer multiplication and division on the HP precision architecture", *IEEE Trans. on Computers* 37(8), 980–990.

MIPS [1997]. *MIPS16 Application Specific Extension Product Description*.

Mitsubishi [1996]. *Mitsubishi 32-Bit Single Chip Microcomputer M32R Family Software Manual* (September).

Muchnick, S. S. [1988]. "Optimizing compilers for SPARC", *Sun Technology* 1:3 (Summer), 64–77.

Seal, D. *Arm Architecture Reference Manual*, 2nd ed, Morgan Kaufmann, 2000.

Silicon Graphics [1996]. *MIPS V Instruction Set*.

Sites, R. L., and R. Witek (Eds.) [1995]. *Alpha Architecture Reference Manual*, 2nd ed., Newton, MA: Digital Press.

Sloss, A. N., D. Symes, and C. Wright, *ARM System Developer's Guide*, San Francisco: Elsevier Morgan Kaufmann, 2004.

Sun Microsystems [1989]. *The SPARC Architectural Manual*, Version 8, Part No. 800-1399-09, August 25.

Sweetman, D. *See MIPS Run*, 2nd ed, Morgan Kaufmann, 2006.

Taylor, G., P. Hilfinger, J. Larus, D. Patterson, and B. Zorn [1986]. "Evaluation of the SPUR LISP architecture," *Proc. 13th Symposium on Computer Architecture* (June), Tokyo.

Ungar, D., R. Blau, P. Foley, D. Samples, and D. Patterson [1984]. "Architecture of SOAR: Smalltalk on a RISC," *Proc. 11th Symposium on Computer Architecture* (June), Ann Arbor, MI, 188–97.

Weaver, D. L. and T. Germond [1994]. *The SPARC Architectural Manual, Version 9*, Prentice Hall, Englewood Cliffs, NJ.

Weiss, S. and J. E. Smith [1994]. *Power and PowerPC*, San Francisco: Morgan Kaufmann.

①

LEGv8 Reference Data

CORE INSTRUCTION SET in Alphabetical Order by Mnemonic

NAME, MNEMONIC		FOR-MAT	OPCODE (9) (Hex)	OPERATION (in Verilog)	Notes
ADD	ADD	R	458	R[Rd] = R[Rn] + R[Rm]	
ADD Immediate	ADDI	I	488-489	R[Rd] = R[Rn] + ALUImm	(2,9)
ADD Immediate & Set flags	ADDIS	I	588-589	R[Rd] , FLAGS = R[Rn] + ALUImm	(1,2,9)
ADD & Set flags	ADDS	R	558	R[Rd] , FLAGS = R[Rn] + R[Rm]	(1)
AND	AND	R	450	R[Rd] = R[Rn] & R[Rm]	
AND Immediate	ANDI	I	490-491	R[Rd] = R[Rn] & ALUImm	(2,9)
AND Immediate & Set flags	ANDIS	I	790-791	R[Rd] , FLAGS = R[Rn] & ALUImm	(1,2,9)
AND & Set flags	ANDS	R	750	R[Rd] , FLAGS = R[Rn] & R[Rm]	(1)
Branch	B	B	0A0-0BF	PC = PC + BranchAddr	(3,9)
Branch conditionally	B.cond	CB	2A0-2A7	if(FLAGS==cond) PC = PC + CondBranchAddr	(4,9)
Branch with Link	BL	B	4A0-4BF	R[30] = PC + 4; PC = PC + BranchAddr	(3,9)
Branch to Register	BR	R	6B0	PC = R[Rt]	
Compare & Branch if Not Zero	CBNZ	CB	5A8-5AF	if(R[Rt]!=0) PC = PC + CondBranchAddr	(4,9)
Compare & Branch if Zero	CBZ	CB	5A0-5A7	if(R[Rt]==0) PC = PC + CondBranchAddr	(4,9)
Exclusive OR	EOR	R	650	R[Rd] = R[Rn] ^ R[Rm]	
Exclusive OR Immediate	EORI	I	690-691	R[Rd] = R[Rn] ^ ALUImm	(2,9)
LoaD Register Unscaled offset	LDUR	D	7C2	R[Rt] = M[R[Rn] + DTAddr]	(5)
LoaD Byte Unscaled offset	LDURB	D	1C2	R[Rt]={56'b0, M[R[Rn] + DTAddr](7:0)}	(5)
LoaD Half Unscaled offset	LDURH	D	3C2	R[Rt]={48'b0, M[R[Rn] + DTAddr] (15:0)}	(5)
LoaD Signed Word Unscaled offset	LDURSW	D	5C4	R[Rt] ={ 32{ M[R[Rn] + DTAddr] [31]}, M[R[Rn] + DTAddr] (31:0)}	(5)
LoaD eXclusive Register	LDXR	D	642	R[Rd] = M[R[Rn] + DTAddr]	(5,7)
Logical Shift Left	LSL	R	69B	R[Rd] = R[Rn] << shamt	
Logical Shift Right	LSR	R	69A	R[Rd] = R[Rn] >>> shamt	
MOVe wide with Keep	MOVK	IM	794-797	R[Rd] (Instruction[22:21]*16: Instruction[22:21]*16-15) = MOVImm	(6,9)
MOVe wide with Zero	MOVZ	IM	694-697	R[Rd] = { MOVImm << (Instruction[22:21]*16) }	(6,9)
Inclusive OR	ORR	R	550	R[Rd] = R[Rn] | R[Rm]	
Inclusive OR Immediate	ORRI	I	590-591	R[Rd] = R[Rn] | ALUImm	(2,9)
STore Register Unscaled offset	STUR	D	7C0	M[R[Rn] + DTAddr] = R[Rt]	(5)
STore Byte Unscaled offset	STURB	D	1C0	M[R[Rn] + DTAddr](7:0) = R[Rt](7:0)	(5)
STore Half Unscaled offset	STURH	D	3C0	M[R[Rn] + DTAddr](15:0) = R[Rt](15:0)	(5)
STore Word Unscaled offset	STURW	D	5C0	M[R[Rn] + DTAddr](31:0) = R[Rt](31:0)	(5)
STore eXclusive Register	STXR	D	640	M[R[Rn] + DTAddr] = R[Rt]; R[Rm] = (*atomic*) ? 0 : 1	(5,7)
SUBtract	SUB	R	658	R[Rd] = R[Rn] – R[Rm]	
SUBtract Immediate	SUBI	I	688-689	R[Rd] = R[Rn] – ALUImm	(2,9)
SUBtract Immediate & Set flags	SUBIS	I	788-789	R[Rd] , FLAGS = R[Rn] – ALUImm	(1,2,9)
SUBtract & Set flags	SUBS	R	758	R[Rd] , FLAGS = R[Rn] – R[Rm]	(1)

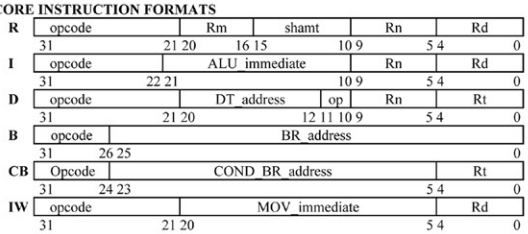
(1) FLAGS are 4 condition codes set by the ALU operation: Negative, Zero, oVerflow, Carry
(2) ALUImm = { 52'b0, ALU_immediate }
(3) BranchAddr = { 36{BR_address [25]}, BR_address, 2'b0 }
(4) CondBranchAddr = { 43{COND_BR_address [25]}, COND_BR_address, 2'b0 }
(5) DTAddr = { 55{DT_address [8]}, DT_address }
(6) MOVImm = { 48'b0, MOV_immediate }
(7) Atomic test&set pair; R[Rm] = 0 if pair atomic, 1 if not atomic
(8) Operands considered unsigned numbers (vs. 2's complement)
(9) Since I, B, and CB instruction formats have opcodes narrower than 11 bits, they occupy a range of 11-bit opcodes
(10) If neither is operand a NaN and Value1 == Value2, FLAGS = 4'b0110;
If neither is operand a NaN and Value1 < Value2, FLAGS = 4'b1000;
If neither is operand a NaN and Value1 > Value2, FLAGS = 4'b0010;
If an operand is a Nan, operands are unordered

ARITHMETIC CORE INSTRUCTION SET

②

NAME, MNEMONIC		FOR-MAT	OPCODE/ SHAMT (Hex)	OPERATION (in Verilog)	Notes
Floating-point ADD Single	FADDS	R	0F1 / 0A	S[Rd] = S[Rn] + S[Rm]	
Floating-point ADD Double	FADDD	R	0F3 / 0A	D[Rd] = D[Rn] + D[Rm]	
Floating-point CoMPare Single	FCMPS	R	0F1 / 08	FLAGS = (S[Rn] vs S[Rm])	(1,10)
Floating-point CoMPare Double	FCMPD	R	0F3 / 08	FLAGS = (D[Rn] vs D[Rm])	(1,10)
Floating-point DIVide Single	FDIVS	R	0F1 / 06	S[Rd] = S[Rn] / S[Rm]	
Floating-point DIVide Double	FDIVD	R	0F3 / 06	D[Rd] = D[Rn] / D[Rm]	
Floating-point MULtiply Single	FMULS	R	0F1 / 02	S[Rd] = S[Rn] * S[Rm]	
Floating-point MULtiply Double	FMULD	R	0F3 / 02	D[Rd] = D[Rn] * D[Rm]	
Floating-point SUBtract Single	FSUBS	R	0F1 / 0E	S[Rd] = S[Rn] – S[Rm]	
Floating-point SUBtract Double	FSUBD	R	0F3 / 0E	D[Rd] = D[Rn] – D[Rm]	
LoaD Single floating-point	LDURS	R	7C2	S[Rt] = M[R[Rn] + DTAddr]	(5)
LoaD Double floating-point	LDURD	R	7C0	D[Rt] = M[R[Rn] + DTAddr]	(5)
MULtiply	MUL	R	4D8 / 1F	R[Rd] = (R[Rn] * R[Rm]) (63:0)	
Signed DIVide	SDIV	R	4D6 / 02	R[Rd] = R[Rn] / R[Rm]	
Signed MULtiply High	SMULH	R	4DA	R[Rd] = (R[Rn] * R[Rm]) (127:64)	
STore Single floating-point	STURS	R	7E2	M[R[Rn] + DTAddr] = S[Rt]	(5)
STore Double floating-point	STURD	R	7E0	M[R[Rn] + DTAddr] = D[Rt]	(5)
Unsigned DIVide	UDIV	R	4D6 / 03	R[Rd] = R[Rn] / R[Rm]	(8)
Unsigned MULtiply High	UMULH	R	4DE	R[Rd] = (R[Rn] * R[Rm]) (127:64)	(8)

CORE INSTRUCTION FORMATS



PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
CoMPare	CMP	FLAGS = R[Rn] – R[Rm]
CoMPare Immediate	CMPI	FLAGS = R[Rn] – ALUImm
LoaD Address	LDA	R[Rd] = R[Rn] + DTAddr
MOVe	MOV	R[Rd] = R[Rn]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
X0 – X7	0-7	Arguments / Results	No
X8	8	Indirect result location register	No
X9 – X15	9-15	Temporaries	No
X16 (IP0)	16	May be used by linker as a scratch register; other times used as temporary register	No
X17 (IP1)	17	May be used by linker as a scratch register; other times used as temporary register	No
X18	18	Platform register for platform independent code; otherwise a temporary register	No
X19-X27	19-27	Saved	Yes
X28 (SP)	28	Stack Pointer	Yes
X29 (FP)	29	Frame Pointer	Yes
X30 (LR)	30	Return Address	Yes
XZR	31	The Constant Value 0	N.A.

OPCODES IN NUMERICAL ORDER BY OPCODE ③

Instruction Mnemonic	Format	Opcode Width (bits)	Opcode Binary	Shamt Binary	11-bit Opcode Range (1) Start (Hex)	End (Hex)
B	B	6	000101		0A0	0BF
FMULS	R	11	00011110001	000010	0F1	
FDIVS	R	11	00011110001	000110	0F1	
FCMPS	R	11	00011110001	001000	0F1	
FADDS	R	11	00011110001	001010	0F1	
FSUBS	R	11	00011110001	001110	0F1	
FMULD	R	11	00011110011	000010	0F3	
FDIVD	R	11	00011110011	000110	0F3	
FCMPD	R	11	00011110011	001000	0F3	
FADDD	R	11	00011110011	001010	0F3	
FSUBD	R	11	00011110011	001110	0F3	
STURB	D	11	00111000000		1C0	
LDURB	D	11	00111000010		1C2	
B.cond	CB	8	01010100		2A0	2A7
STURH	D	11	01111000000		3C0	
LDURH	D	11	01111000010		3C2	
AND	R	11	10001010000		450	
ADD	R	11	10001011000		458	
ADDI	I	10	1001000100		488	489
ANDI	I	10	1001001000		490	491
BL	B	6	100101		4A0	4BF
SDIV	R	11	10011010110	000010	4D6	
UDIV	R	11	10011010110	000011	4D6	
MUL	R	11	10011011000	011111	4D8	
SMULH	R	11	10011011010		4DA	
UMULH	R	11	10011011110		4DE	
ORR	R	11	10101010000		550	
ADDS	R	11	10101011000		558	
ADDIS	I	10	1011000100		588	589
ORRI	I	10	1011001000		590	591
CBZ	CB	8	10110100		5A0	5A7
CBNZ	CB	8	10110101		5A8	5AF
STURW	D	11	10111000000		5C0	
LDURSW	D	11	10111000100		5C4	
STURS	R	11	10111100000		5E0	
LDURS	R	11	10111100010		5E2	
STXR	D	11	11001000000		640	
LDXR	D	11	11001000010		642	
EOR	R	11	11001010000		650	
SUB	R	11	11001011000		658	
SUBI	I	10	1101000100		688	689
EORI	I	10	1101001000		690	691
MOVZ	IM	9	110100101		694	697
LSR	R	11	11010011010		69A	
LSL	R	11	11010011011		69B	
BR	R	11	11010110000		6B0	
ANDS	R	11	11101010000		750	
SUBS	R	11	11101011000		758	
SUBIS	I	10	1111000100		788	789
ANDIS	I	10	1111001000		790	791
MOVK	IM	9	111100101		794	797
STUR	D	11	11111000000		7C0	
LDUR	D	11	11111000010		7C2	
STURD	R	11	11111100000		7E0	
LDURD	R	11	11111100010		7E2	

(1) Since I, B, and CB instruction formats have opcodes narrower than 11 bits, they occupy a range of 11-bit opcodes, e.g., the 6-bit B format occupies 32 (2^5) 11-bit opcodes.

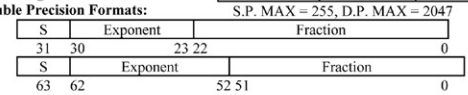
IEEE 754 FLOATING-POINT STANDARD ④

$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$
where Single Precision Bias = 127,
Double Precision Bias = 1023

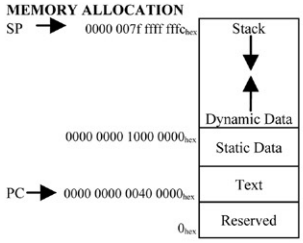
IEEE 754 Symbols

Exponent	Fraction	Object
0	0	± 0
0	≠ 0	± Denorm
1 to MAX - 1	anything	± F1. Pt. Num.
MAX	0	± ∞
MAX	≠ 0	NaN

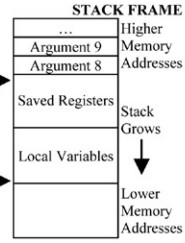
IEEE Single Precision and Double Precision Formats: S.P. MAX = 255, D.P. MAX = 2047



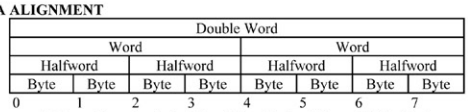
MEMORY ALLOCATION



STACK FRAME

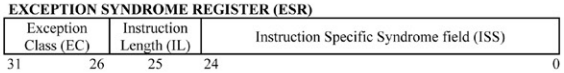


DATA ALIGNMENT



Value of three least significant bits of byte address (Big Endian)

EXCEPTION SYNDROME REGISTER (ESR)



EXCEPTION CLASS

EC	Class	Cause of Exception	Number	Name	Cause of Exception
0	Unknown	Unknown	34	PC	Misaligned PC exception
7	SIMD	SIMD/FP registers disabled	36	Data	Data Abort
14	FPE	Illegal Execution State	40	FPE	Floating-point exception
17	Sys	Supervisor Call Exception	52	WPT	Data Breakpoint exception
32	Instr	Instruction Abort	56	BKPT	SW Breakpoint Exception

SIZE PREFIXES AND SYMBOLS

SIZE	PREFIX	SYMBOL	SIZE	PREFIX	SYMBOL
10^3	Kilo-	K	2^{10}	Kibi-	Ki
10^6	Mega-	M	2^{20}	Mebi-	Mi
10^9	Giga-	G	2^{30}	Gibi-	Gi
10^{12}	Tera-	T	2^{40}	Tebi-	Ti
10^{15}	Peta-	P	2^{50}	Pebi-	Pi
10^{18}	Exa-	E	2^{60}	Exbi-	Ei
10^{21}	Zetta-	Z	2^{70}	Zebi-	Zi
10^{24}	Yotta-	Y	2^{80}	Yobi-	Yi
10^{-3}	milli-	m	10^{-15}	femto-	f
10^{-6}	micro-	μ	10^{-18}	atto-	a
10^{-9}	nano-	n	10^{-21}	zepto-	z
10^{-12}	pico-	p	10^{-24}	yocto-	y

LEGv8 Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together

COMPUTER ORGANIZATION AND DESIGN

THE HARDWARE/SOFTWARE INTERFACE

ARM® EDITION

DAVID A. PATTERSON | JOHN L. HENNESSY

"We are delighted to support this edition of *Computer Organization and Design*, a textbook that has inspired generations of computer scientists and engineers. This new edition will enable students and engineers worldwide to learn and master the theory, while practicing with the most widely used processor architecture in mobile computing today."

— Dr. Khaled Benkrid, Worldwide University Program Manager, ARM Ltd.

The new ARM Edition of *Computer Organization and Design* has been updated to feature a subset of the ARMv8-A architecture, which is used to present the fundamentals of hardware technologies, assembly language, computer arithmetic, pipelining, memory hierarchies, and I/O.

With the post-PC era now upon us, *Computer Organization and Design* moves forward to explore this generational change with examples, exercises, and material highlighting the emergence of mobile computing and the Cloud. Updated content featuring tablet computers, Cloud infrastructure, and the ARM (mobile computing devices) and x86 (Cloud computing) architectures is included.

An online companion Web site provides links to a free Community Edition of the ARM DS-5 professional software suite which contains an ARMv8-A (64-bit) architecture simulator, as well as additional advanced content for further study, appendices, glossary, references, and recommended reading.

ARM EDITION FEATURES

- Covers parallelism in depth with examples and content highlighting parallel hardware and software topics
- Features the Intel Core i7, ARM Cortex-A53, and NVIDIA Fermi GPU as real-world examples throughout the book
- Adds a new concrete example, "Going Faster," to demonstrate how understanding hardware can inspire software optimizations that improve performance by 200X
- Discusses and highlights the "Eight Great Ideas" of computer architecture: Performance via Parallelism; Performance via Pipelining; Performance via Prediction; Design for Moore's Law; Hierarchy of Memories; Abstraction to Simplify Design; Make the Common Case Fast; and Dependability via Redundancy
- Includes a full set of updated and improved exercises

ABOUT THE AUTHORS

David A. Patterson
Pardee Chair of Computer Science
University of California at Berkeley

John L. Hennessy
President
Stanford University

COMPUTER SYSTEMS DESIGN
COMPUTER HARDWARE

MORGAN KAUFMANN PUBLISHERS
AN IMPRINT OF ELSEVIER
elsevier.com

ISBN 978-0-12-801733-3
9 780128 017333